

MS4215: Advanced Data Analysis

Week 3A: MLR - Multicollinearity

Associate Professor Sinéad Moylett

September 21, 2026

Learning outcomes

- Explain what multicollinearity is and describe how it arises from correlated predictors
- Identify the four consequences of severe multicollinearity: coefficient instability, inflated standard errors, sequential sums of squares (SS) dependence, and changing significance
- Detect multicollinearity using pairwise correlations, the variance inflation factor (VIF), and the condition number
- Apply a remedy for multicollinearity by removing redundant predictors and comparing model fit before and after

What is Multicollinearity?

Where we left off

Week 02B: Newspaper predictor

In the Advertising dataset, **newspaper** was:

- **Significant** in SLR ($p = 0.001$)
- **Not significant** in MLR ($p = 0.86$)

Why? Because **newspaper** and **radio** are correlated ($r = 0.35$). In the SLR, newspaper was taking credit for radio's effect.

That was a **mild** case.

Today's question

What happens when predictor correlations are much stronger?

- $r = 0.87, 0.90, 0.95$ between predictors
- $VIF = 10, 15, 20$

The same mechanism operates, but the damage is far greater.

Definition

Multicollinearity occurs when two or more predictor variables are highly correlated with each other.

Note

A common rule of thumb: multicollinearity is flagged when a pairwise correlation between predictors exceeds $|r| > 0.9$.

VIF provides a more formal measure. We'll cover this in today's lecture.

Two types:

Data-based multicollinearity

Arises from observational data where we cannot control which variables move together.

Example: engine size, horsepower, and weight in the **Cars** data = larger engines always have more horsepower and greater vehicle weight.

Structural multicollinearity

Arises from model terms we create ourselves.

Example: including both x and x^2 as predictors. By construction, these are correlated.

Addressed in **Week 03B** (polynomial regression).

The mathematical mechanism

Recall the ordinary least squares (OLS) estimator and its variance:

$$\mathbf{b} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y} \quad \text{Var}(\mathbf{b}) = \sigma^2(\mathbf{X}'\mathbf{X})^{-1}$$

When two or more predictors are highly correlated:

$$\det(\mathbf{X}'\mathbf{X}) \approx 0 \quad \Rightarrow \quad \mathbf{X}'\mathbf{X} \text{ is near-singular}$$

Inversion of a near-singular matrix is numerically unstable: the diagonal elements of $(\mathbf{X}'\mathbf{X})^{-1}$ become very large.

$$\text{Var}(b_j) = \sigma^2 [(\mathbf{X}'\mathbf{X})^{-1}]_{jj} \longrightarrow \text{inflates}$$

Warning

Inflated variances mean inflated standard errors, which shrink t-statistics and widen confidence intervals, making it harder to detect genuine effects.

A matrix illustration (1/2): Well-conditioned

Two predictors, two observations: $X_1 = (7, 12)$, $X_2 = (3, 6)$.

Recall: for a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$, the inverse is $\frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$.

Step 1: Compute $\mathbf{X}'\mathbf{X}$ (dot products of predictor columns)

$$\mathbf{C} = \mathbf{X}'\mathbf{X} = \begin{pmatrix} 7^2 + 12^2 & 7 \times 3 + 12 \times 6 \\ 7 \times 3 + 12 \times 6 & 3^2 + 6^2 \end{pmatrix} = \begin{pmatrix} 193 & 93 \\ 93 & 45 \end{pmatrix}$$

Step 2: Determinant

$$\det(\mathbf{C}) = 193 \times 45 - 93 \times 93 = 8685 - 8649 = 36$$

Step 3: Inverse

$$\mathbf{C}^{-1} = \frac{1}{36} \begin{pmatrix} 45 & -93 \\ -93 & 193 \end{pmatrix} = \begin{pmatrix} 1.25 & -2.58 \\ -2.58 & 5.36 \end{pmatrix}$$

Diagonal elements are modest. Estimation is stable.

A matrix illustration (2/2): Near-singular

Now change X_1 from (7, 12) to (6.02, 12).

Note: $6.02/12 \approx 0.50$ and $3/6 = 0.50$ = the columns are now almost proportional ($X_1 \approx 2X_2$).

Step 1: Recompute $\mathbf{X}'\mathbf{X}$

$$\mathbf{C} = \begin{pmatrix} 6.02^2 + 12^2 & 6.02 \times 3 + 12 \times 6 \\ 6.02 \times 3 + 12 \times 6 & 3^2 + 6^2 \end{pmatrix} = \begin{pmatrix} 180.24 & 90.06 \\ 90.06 & 45 \end{pmatrix}$$

Step 2: Determinant

$$\det(\mathbf{C}) = 180.24 \times 45 - 90.06^2 = 8110.8 - 8110.8 \approx 0.014$$

Step 3: Inverse

$$\mathbf{C}^{-1} = \frac{1}{0.014} \begin{pmatrix} 45 & -90.06 \\ -90.06 & 180.24 \end{pmatrix} = \begin{pmatrix} 3125 & -6254 \\ -6254 & 12516 \end{pmatrix}$$

Warning

Changing X_1 from 7 to 6.02 (less than 1%) collapsed $\det(\mathbf{C})$ from 36 to 0.014 and inflated the diagonal elements by a factor of ~2,500.

Variances that were 1.25 and 5.36 are now 3125 and 12516.

Geometric argument: The elongated valley

Uncorrelated predictors

The residual sum of squares (RSS) contours are roughly circular. There is a unique, well-defined minimum. Small changes in data produce small changes in the coefficient estimates.

Correlated predictors

RSS contours form a narrow, elongated valley. Many coefficient pairs give almost identical RSS.

“A small change in the data could cause the least squares estimates to move anywhere along this valley.”

Note

The valley explains why correlated coefficients are **unstable**: The optimum is flat along the valley floor, so the solver has no reason to prefer one point over another.

Effect 1: Coefficients change

Blood Pressure example: $n = 20$. Response: **systolic BP**. Predictors include **Weight** and body surface area (**BSA**), where $r(\text{Weight}, \text{BSA}) = 0.875$.

Model	BSA coefficient
BSA alone	34.44
BSA + Weight	5.83

The coefficient for **BSA** drops by more than 80% when **Weight** enters the model.

Warning

You cannot interpret a coefficient in isolation when predictors are correlated. The coefficient of **BSA** is not a fixed property of the data. It depends on what else is in the model.

Effect 2: Standard errors inflate

Blood Pressure example: $n = 20$. Response: **systolic BP**. Predictors include **Weight** and body surface area (**BSA**), where $r(\text{Weight}, \text{BSA}) = 0.875$.

Predictor	Fitted alone: SE	Fitted with other: SE
BSA	4.69	6.06
Weight	0.093	0.193

BSA 's SE increases by 29%. **Weight** 's SE more than doubles.

Note

Inflated SEs mean t-statistics shrink: $t = b/\text{SE}(b)$.

A predictor may fail to be significant not because it is unrelated to the response, but because a correlated companion has already absorbed the signal.

Effect 3: Sequential SS depend on order

Blood Pressure example: $n = 20$. Response: `systolic BP`. Predictors include `Weight` and body surface area (`BSA`), where $r(\text{Weight}, \text{BSA}) = 0.875$.

`anova()` output:

Term added	Sequential SS
<code>Weight</code> (first)	505.47
<code>BSA Weight</code>	88.43
<code>BSA</code> (first)	—
<code>Weight BSA</code>	2.81

Adding `Weight` after `BSA` explains 88 units; adding `BSA` after `Weight` explains only 2.8 units.

Warning

When predictors are correlated, the sequential sums of squares from `anova()` change depending on the order predictors are entered. There is no unique decomposition of explained variance between correlated predictors.

Effect 4: Significance changes

Blood Pressure example: $n = 20$. Response: **systolic BP**. Predictors include **Weight** and body surface area (**BSA**), where $r(\text{Weight}, \text{BSA}) = 0.875$.

Model	BSA p-value
BSA alone	p = 0.000
BSA + Weight	p = 0.350

BSA is genuinely related to blood pressure. Yet it appears non-significant when **Weight** is already in the model.

Warning

A non-significant predictor in an MLR does not necessarily mean that predictor is unrelated to the response. It may simply be **redundant** given what is already in the model.

What multicollinearity damages, and what it doesn't

What multicollinearity damages

- Individual coefficient estimates = unstable, can change sign
- Standard errors = inflated
- t-tests and p-values = unreliable for individual predictors
- Interpretation = cannot say which predictor is responsible

What is preserved despite multicollinearity

- Fitted values $\hat{y}$
- Overall model predictions
- Global F-test for the whole model
- Prediction MSE on new data

Adding noise to **y** and refitting: coefficient signs may flip, but fitted values barely change.

Warning

Multicollinearity must always be diagnosed, regardless of your modelling goal.

Even in a prediction context, undetected multicollinearity means you cannot trust individual coefficients, cannot interpret the model, and may be misled if the predictor relationships change in new data.

Detecting Multicollinearity

The **longley** dataset

Built into R. Sixteen annual US economic observations from 1947 to 1962.

Variable	Role	Description
Employed	response	Total employment (thousands)
GNP	predictor	Gross national product
Unemployed	predictor	Number unemployed (thousands)
Armed.Forces	predictor	Armed forces size (thousands)
Population	predictor	Non-institutionalised population ≥ 14 years
Year	predictor	Year (1947–1962)

Note

GNP , **Population** , and **Year** all trend upward together over the post-war period, making this a classic demonstration dataset for severe multicollinearity.

Tool 1: Pairwise correlations

First step: Examine the predictor correlation matrix.

```
1 longley_vars <- longley[, c("GNP", "Unemployed",  
2                             "Armed.Forces",  
3                             "Population", "Year")]  
4 round(cor(longley_vars), 2)
```

	GNP	Unemployed	Armed.Forces	Population	Year
GNP	1.00	0.60	0.45	0.99	1.00
Unemployed	0.60	1.00	-0.18	0.69	0.67
Armed.Forces	0.45	-0.18	1.00	0.36	0.42
Population	0.99	0.69	0.36	1.00	0.99
Year	1.00	0.67	0.42	0.99	1.00

```
1 corrplot(cor(longley_vars),  
2          method = "color",  
3          addCoef.col = "white",  
4          tl.cex = 0.8)
```



Note

Three pairs exceed $|r| > 0.9$: **GNP – Population** ($r = 0.99$), **GNP – Year** ($r = 1.00$), and **Population – Year** ($r = 0.99$). These three predictors form a near-perfectly collinear cluster, all trending upward together across the post-war period.

The limitation of pairwise correlations

Pairwise correlations only catch **two-variable** relationships.

The problem

You can have near-multicollinearity from a linear combination of three or more variables, even when no single pair has $|r| > 0.9$.

If $x_3 \approx 2x_1 + x_2$, then $r(x_1, x_3)$ and $r(x_2, x_3)$ may both be only moderate, yet the combination creates severe collinearity.

The solution: Variance Inflation Factor

The Variance Inflation Factor (VIF) explicitly checks whether each predictor can be predicted from all the others combined.

It catches multi-variable collinearity that pairwise correlations miss.

Tool 2: The Variance Inflation Factor

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

where R_j^2 is the R^2 from regressing x_j on all other predictors.

Interpretation:

$$\text{Var}(b_j) = \text{VIF}_j \times \text{Var}(b_j^*)$$

where b_j^* is the coefficient if all predictors were **uncorrelated**.

VIF	Meaning
1	Predictor j is uncorrelated with all others
> 1	Some collinearity present. Variance is inflated by a factor of VIF
>> 1	Severe collinearity

Manual VIF calculation

Blood Pressure example: $n = 20$. Response: **systolic BP**. Predictors include **Weight** and body surface area (**BSA**), where $r(\text{Weight}, \text{BSA}) = 0.875$.

To find VIF for **Weight**: regress **Weight** on all other predictors (**Age**, **BSA**, **Duration**, **Pulse**, **Stress**).

That auxiliary regression gives $R^2_{\text{Weight}} = 0.881$.

$$\text{VIF}_{\text{Weight}} = \frac{1}{1 - 0.881} = \frac{1}{0.119} = 8.4$$

In R, to compute manually for the **longley** data:

```
1 # Auxiliary regression: GNP on all other predictors
2 aux <- lm(GNP ~ Unemployed + Armed.Forces + Population + Year,
3           data = longley)
4 r2_gnp <- summary(aux)$r.squared
5 vif_gnp <- 1 / (1 - r2_gnp)
6 vif_gnp
```

[1] 1034.218

Or: use **car::vif(longley_mod)** to get all VIFs at once.

VIF thresholds

There is no single agreed threshold. Econometrics tends to accept higher VIFs than biostatistics. Rules of thumb vary across disciplines:

Threshold	Interpretation
VIF > 2.5	Some multicollinearity present
VIF > 4	Warrants investigation
VIF > 5	Cause for concern
VIF > 10	Serious = requires action

Note

For this course: VIF > 5 = flag; VIF > 10 = act.

Context matters: a VIF of 3 in a large dataset with precise estimates may be tolerable; a VIF of 3 in a small dataset with borderline significance is more serious.

car::vif() in R

```
1 library(car)
2 vif(longley_mod)
```

	GNP	Unemployed	Armed.Forces	Population	Year
	1034.217736	23.263435	3.152259	225.713446	732.642507

Returns a named vector of VIF values, one per predictor.

Note

`car::vif()` is the primary VIF tool for this course. VIFs in the `longley` model are extreme, far beyond the $VIF > 10$ action threshold.

jtools::summ(): VIFs with coefficients

```
1 library(jtools)
2 s <- summ(longley_mod, vif = TRUE)
3 knitr::kable(s$coefstable, digits = 3)
```

	Est.	S.E.	t val.	p	VIF
(Intercept)	-3449.892	828.242	-4.165	0.002	NA
GNP	-0.032	0.024	-1.321	0.216	1034.218
Unemployed	-0.020	0.004	-5.108	0.000	23.263
Armed.Forces	-0.010	0.002	-5.345	0.000	3.152
Population	-0.078	0.162	-0.480	0.642	225.713
Year	1.814	0.425	4.266	0.002	732.643

Note

`jtools::summ()` produces the same VIF values as `car::vif()` but alongside the coefficient table. Useful when you want to see coefficients, standard errors, and VIFs together in one place.

Tool 3: The condition number

The **condition number** κ measures the overall conditioning of the design matrix:

$$\kappa = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}}$$

where λ are the eigenvalues of the correlation matrix of the predictors.

A large spread between the largest and smallest eigenvalue means the matrix is nearly rank-deficient, i.e., some linear combination of predictors is close to zero.

Note

The condition number catches multicollinearity that involves **combinations of three or more predictors**, which pairwise correlations and sometimes even VIF may understate.

Tool 3: The condition number, in R

```
1 X_mat <- model.matrix(longley_mod)[, -1]
2 kappa(cor(X_mat))
```

```
[1] 5562.731
```

K	Interpretation
< 10	Acceptable
10–30	Moderate concern
> 30	Severe multicollinearity

Note

For the **longley** model, the condition number flags severe multicollinearity, consistent with the VIF results.

How serious can it get? The ACL example

Allen Cognitive Level study

Initial sample: $n = 23$

Predictors Vocab and Abstract:

- $r(\text{Vocab}, \text{Abstract}) = 0.990$
- $\text{VIF}(\text{Vocab}) = 49.3$
- $\text{VIF}(\text{Abstract}) = 50.6$

Extreme multicollinearity: the two predictors are almost identical.

After collecting more data: $n = 69$

- $r(\text{Vocab}, \text{Abstract}) = 0.698$
- $\text{VIF}(\text{Vocab}) = 2.09$
- $\text{VIF}(\text{Abstract}) = 2.17$

Now acceptable.

Note

Multicollinearity is partly a **data problem**, not purely a model problem. The initial sample didn't include enough subjects where **Vocab** and **Abstract** diverged. More varied data can reduce collinearity without changing the model structure.

Cars: Case Study

The Cars dataset

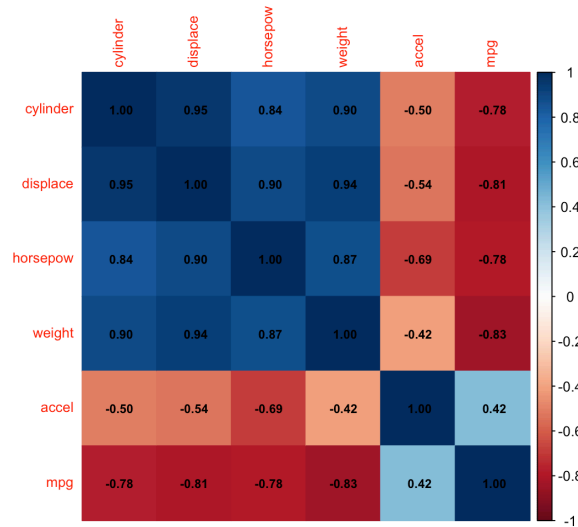
404 classic cars from the UCI Machine Learning Repository.

Variable	Role	Description
<code>mpg</code>	response	Fuel efficiency (miles per gallon)
<code>cylinder</code>	predictor	Number of cylinders
<code>displace</code>	predictor	Engine displacement (cc)
<code>horsepow</code>	predictor	Horsepower
<code>weight</code>	predictor	Vehicle weight (kg)
<code>accel</code>	predictor	0–60 mph time (seconds)
<code>model_ye</code>	predictor	Model year (e.g. 70 = 1970)
<code>origin</code>	predictor	1 = American, 2 = European, 3 = Japanese

Missing data: 8 NAs in `mpg` , 6 in `horsepow` , remove with `na.omit()` before fitting.

Cars: Correlation matrix

```
1 numeric_vars <- cars_clean[, c("cylinder","displace","horsepow",  
2                               "weight","accel","mpg")]  
3 corplot(cor(numeric_vars, use = "pairwise.complete.obs"),  
4         method = "color",  
5         addCoef.col = "black",  
6         tl.cex = 0.8,  
7         number.cex = 0.7)
```



Key values:

Pair	r
cylinder–displace	0.95
displace–weight	0.94
cylinder–weight	0.90
horsepow–weight	0.87
cylinder–horsepow	0.84

Fitting the full model

```
1 cars_full <- lm(mpg ~ cylinder + displac + horsepower + weight + accel,  
2                 data = cars_clean)  
3 summary(cars_full)
```

Call:

```
lm(formula = mpg ~ cylinder + displac + horsepower + weight +  
    accel, data = cars_clean)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-11.6071	-2.8310	-0.3178	2.2748	16.3328

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	46.2255126	2.6748981	17.281	< 2e-16	***
cylinder	-0.4320086	0.4131906	-1.046	0.296	
displace	0.0014075	0.0094044	0.150	0.881	
horsepow	-0.0417969	0.0172670	-2.421	0.016	*
weight	-0.0054154	0.0008849	-6.120	2.32e-09	***
accel	-0.0116233	0.1288570	-0.090	0.928	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Warning

Three of five predictors non-significant in a highly significant model ($F = 185.8$, $p < 0.001$). This is a classic multicollinearity symptom.

Red flags in the output

Two things to check: coefficient signs and individual significance.

What we expect

The four engine-power predictors should have **negative** coefficients with mpg:

- More cylinders → worse fuel economy
- Bigger displacement → worse
- More horsepower → worse
- More weight → worse

accel (0–60 time): $r = 0.42$ with mpg suggests a positive marginal relationship, but should the coefficient be positive or negative?

What the full model gives

- **cylinder** = -0.43 , non-significant
- **displace** = $+0.0014$: wrong sign (expected negative)
- **horsepow** = -0.042 , significant
- **weight** = -0.0054 , significant
- **accel** = -0.012 : negative and non-significant ($p = 0.928$)

Warning

A coefficient with the **wrong sign** is one of the clearest indicators of multicollinearity. The model cannot separate the individual contributions of variables that move together.

Checking VIFs

```
1 library(car)
2 vif(cars_full)
```

```
cylinder  displac  horsepower  weight  accel
10.664777 20.584311  9.313529 12.204954  2.705196
```

Applying course thresholds (VIF > 5 = flag, VIF > 10 = act):

- **cylinder** = 10.66: act (VIF > 10)
- **displace** = 20.58: act, extremely severe
- **horsepow** = 9.31: flag (VIF > 5)
- **weight** = 12.20: act (VIF > 10)
- **accel** = 2.71: clean

displace is the worst: it is the predictor most correlated with the others ($r = 0.95$ with **cylinder**, $r = 0.94$ with **weight**). **accel** is the only predictor outside the engine-size cluster.

What do the correlated predictors represent?

The physical reality

Cylinder count, engine displacement, and horsepower all measure the **same underlying thing**: engine size and power.

In the real world:

- Larger engines → more cylinders
- More cylinders → greater displacement
- Greater displacement → more horsepower
- More power → heavier vehicle

These variables are tied together **by physics**, not just by this dataset.

The modelling consequence

Including all four in one model asks the data to separate effects that are **inherently inseparable** in the real world.

The model cannot answer: *“If we held displacement constant but increased horsepower, what would happen to mpg?”*: because that combination barely exists in the data.

Strategy for reduction

Which predictor to keep from the correlated cluster? Three criteria:

1. Strongest marginal correlation with mpg?

From the correlation matrix: **weight** $r = -0.83$, **displace** $r = -0.81$, **cylinder** $r = -0.78$, **horsepow** $r = -0.78$. **Weight** is marginally highest, but all four are close.

2. Lowest VIF in the full model?

horsepow has the lowest VIF of the four (9.31), compared to **cylinder** (10.66), **weight** (12.20), and **displace** (20.58). It is the least collinear representative of the engine cluster.

3. Most directly interpretable?

Horsepower is widely understood as a measure of engine performance, making it the most meaningful predictor to retain.

Decision: keep **horsepow** and **accel** (already clean, $VIF = 2.71$); remove **cylinder**, **displace**, and **weight**.

Cars: The reduced model

```
1 cars_reduced <- lm(mpg ~ horsepower + accel, data = cars_clean)
2 summary(cars_reduced)
```

Call:

```
lm(formula = mpg ~ horsepower + accel, data = cars_clean)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.1738	-3.1936	-0.6667	2.6887	17.1127

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	52.836055	2.582762	20.457	< 2e-16 ***
horsepower	-0.190277	0.008668	-21.953	< 2e-16 ***
accel	-0.613449	0.120018	-5.111	5.04e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.742 on 387 degrees of freedom

Multiple R-squared: 0.6332, Adjusted R-squared: 0.6313

F-statistic: 334.1 on 2 and 387 DF, p-value: < 2.2e-16

Note

The difference is that here it is highly significant ($p < 0.001$) and has a clear conditional interpretation: controlling for **horsepower**, a car that takes longer to reach 60 mph is likely heavier or less aerodynamic, and both characteristics are associated with lower **mpg**.

In the full model the negative sign was an artefact of multicollinearity. Here it is a stable, significant conditional effect.

Both predictors highly significant.

horsepower has the expected negative sign.

accel is still negative (-0.61), which looks like the same wrong sign flagged in the full model.

Reduced model VIFs

```
1 vif(cars_reduced)
```

```
horsepow  accel  
1.88627  1.88627
```

Predictor	VIF (full model)	VIF (reduced model)
cylinder	10.66	<i>removed</i>
displace	20.58	<i>removed</i>
horsepow	9.31	1.89
weight	12.20	<i>removed</i>
accel	2.71	1.89

Note

Both VIFs are now well below 5. The multicollinearity problem is resolved. Comparing the maximum VIF: 20.58 in the full model versus 1.89 in the reduced model.

Comparing full and reduced models

	Full model	Reduced model
Predictors	5	2
R ²	0.71	0.63
Adj R ²	0.70	0.63
F-statistic	185.8	334.1
Max VIF	20.58	1.89
Unexplained wrong-sign coefficients?	Yes (displace, accel)	No
All significant?	No (3 of 5)	Yes
Interpretable?	No	Yes

R² drops from 0.71 to 0.63, a cost of 8 percentage points. The reduced model is simpler, cleaner, and interpretable.

Partial F-test

Were the dropped predictors (`cylinder` , `displace` , `weight`) needed, given `horsepow` and `accel` are already in the model?

```
1 anova(cars_reduced, cars_full)
```

Analysis of Variance Table

Model 1: mpg ~ horsepow + accel

Model 2: mpg ~ cylinder + displace + horsepow + weight + accel

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	387	8703.1				
2	384	6941.0	3	1762.1	32.495	< 2.2e-16 ***

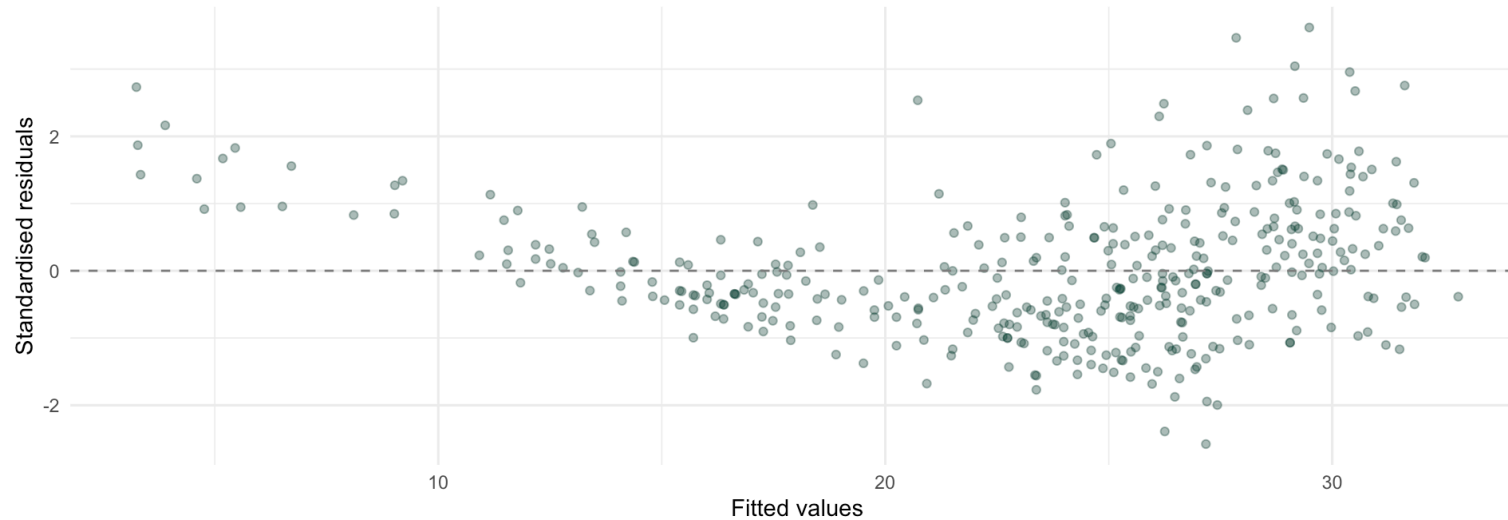
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

$F(3, 384) = 32.5, p < 0.001$.

The dropped predictors are jointly significant, they do explain additional variance. However, this does not mean they should be retained.

Their individual coefficients had wrong signs, p-values were unreliable, and VIFs were extreme. Removing them restores interpretability at a cost of 8 percentage points of R^2 .

Cars: Residual diagnostics



Warning

Residuals vs fitted shows a **fan pattern**: variance of residuals increases for higher fitted mpg values. This is **heteroscedasticity**: the equal-variance (E in LINE) assumption is violated.

This is a separate problem from multicollinearity, revealed after we resolved the collinearity. A transformation may help, which we will address in **Week 03B**.

Remedies for Multicollinearity

Overview of remedies

Four options, roughly in order of invasiveness:

1. Remove

Keep only the best representative from each correlated cluster.

Simplest fix; loses some information.

2. Combine

Create a composite predictor (average, index, or PCA score) from the correlated variables.

Retains information; harder to interpret.

3. Collect more data

Especially data collected under conditions where the correlated predictors vary more independently.

Addresses the root cause.

4. Regularisation

Ridge regression shrinks coefficients and handles near-singular $X'X$ directly.

Preview for a later lecture.

Remedy 1: Remove collinear predictors

“The simplest way to cure the problem of multicollinearity is to drop one of the problematic variables from the regression. Usually little information is lost by doing so, since the two problematic variables carry similar information.”

How to choose which to keep:

- Highest $|r|$ with the response (check `cor()` output)
- Lowest VIF after removal
- Most substantively meaningful or interpretable in context

Cars example: dropping `cylinder`, `displace`, `weight` and keeping `horsepow` costs 8 percentage points of R^2 but resolves all VIF problems and restores interpretability.

Remedy 2: Combine predictors

“Another solution is to combine the collinear variables together into a single predictor. For instance, we might take the average of standardised versions of limit and rating in order to create a new variable.”

In the Cars data: Could create a single composite **engine_power** from **cylinder**, **displace**, **horsepow**, **weight**, e.g. first principal component of the four.

Pros

- Retains information from all correlated predictors
- No information discarded

Cons

- Harder to interpret the composite
- Individual predictor meaning is lost
- Requires explaining what the composite represents

Principal Components Analysis (PCA) does this systematically.

Remedy 3: Collect more data

ACL example revisited

n = 23

$r(\text{Vocab}, \text{Abstract}) = 0.990$

VIF = 49.3 and 50.6

Unusable: predictors are almost identical in this sample.

n = 69

$r(\text{Vocab}, \text{Abstract}) = 0.698$

VIF = 2.09 and 2.17

Acceptable: the larger, more varied sample allows the predictors to vary more independently.

Note

The initial sample didn't include enough subjects where **Vocab** and **Abstract** diverged. If you can redesign your sampling strategy to vary predictors more independently, collinearity can often be reduced without changing the model structure.

Caveat: In observational studies (like the **Cars** data), you cannot redesign what data were collected. Remedy 3 is most applicable when the study design is still under your control.

Remedy 4: Regularisation (preview)

Ridge regression modifies OLS by adding a penalty to the loss function:

$$\text{Minimise } (\mathbf{Y} - \mathbf{X}\mathbf{b})'(\mathbf{Y} - \mathbf{X}\mathbf{b}) + \lambda \|\mathbf{b}\|^2$$

This constrains coefficient sizes and stabilises estimates even when $\mathbf{X}'\mathbf{X}$ is near-singular.

The effect

- Keeps all predictors in the model
- Shrinks correlated coefficients toward each other
- As $\lambda \rightarrow 0$: recovers OLS
- As $\lambda \rightarrow \infty$: all coefficients shrink to zero

When to use it

- When you want to keep all predictors but stabilise the estimates
- When the goal is prediction rather than interpretation
- When dropping predictors is theoretically unjustified

Covered fully in **Week 8**.

Choosing the right remedy

Situation	Best remedy
One predictor clearly redundant (high VIF, low marginal r with y)	Remove it
All correlated predictors theoretically important	Combine (PCA or average)
Sample is small or unvaried; budget allows more data collection	Collect more data
Goal is prediction; want to retain all predictors	Regularisation (ridge)
Structural collinearity from polynomial terms	Centering (Week 03B)

Note

These are not mutually exclusive. In practice, you might remove the most extreme predictor, combine a cluster of two remaining ones, and then verify with VIFs. Iterating until all VIFs are below your threshold.

The **Cars** solution used **Remedy 1** (remove): straightforward because the four power variables were highly redundant and horsepower was the most interpretable representative.

When multicollinearity is (and isn't) a problem

Matters a lot

When the goal is to interpret individual coefficients or test which predictors are significant:

- Unstable coefficients directly undermine inference
- Inflated SEs make t-tests unreliable
- Wrong signs make the model uninterpretable
- Sequential SS are order-dependent

Most modelling in this course aims at **interpretation**.

Matters less

When the goal is prediction on new data from the same population:

- Fitted values $\hat{y}$ are stable even when coefficients are not
- Prediction MSE barely changes under collinearity
- The overall F-test remains valid
- Out-of-sample RMSE is unaffected

Adding random noise to the response and refitting: coefficient signs may flip, but fitted values barely change.

The full detection and remediation workflow

1. Fit the full model with all candidate predictors
2. Check pairwise correlations: `cor()` and `corrplot()` : flag $|r| > 0.9$
3. Compute VIFs: `car::vif()` : flag $VIF > 5$, act on $VIF > 10$
4. Compute condition number: `kappa(cor(X_matrix))` : flag $\kappa > 30$
5. For each flagged cluster: choose one representative to keep (highest $|r|$ with response, or most interpretable)
6. Refit reduced model, re-check VIFs: all should be < 5
7. Run diagnostic plots: does the model now satisfy LINE assumptions?
8. Use `anova(reduced, full)` to formally test whether the dropped predictors were needed

R functions

Function	Package	Purpose
<code>cor(X)</code>	base	Correlation matrix for predictors
<code>corrplot(M)</code>	corrplot	Visual correlation matrix
<code>vif(model)</code>	car	Variance inflation factors
<code>kappa(cor(X))</code>	base	Condition number
<code>anova(reduced, full)</code>	base	Partial F-test after removing predictors

Summary

Multicollinearity (high correlation among predictors) causes $\mathbf{X}'\mathbf{X}$ to become near-singular, inflating standard errors and destabilising coefficient estimates.

Four effects:

- Coefficients change with model composition
- Standard errors inflate, shrinking t-statistics
- Sequential SS from `anova()` are order-dependent
- Genuinely significant predictors may appear non-significant

Detection: pairwise $|r| > 0.9$; $VIF > 5$ (flag), $VIF > 10$ (act); condition number $\kappa > 30$

Remedies: remove redundant predictors; combine into a composite; collect more varied data; use regularisation (ridge)

Key asymmetry: collinearity harms **interpretation** far more than **prediction**

Cars example: VIFs up to 20.58 in the full model $\rightarrow$ 1.89 after removing collinear predictors. Residuals then reveal heteroscedasticity $\rightarrow$ **Week 03B**